

开发初始化（从设计到工程）

工程架构：

代码块

```
1  docforge/
2    app/
3      main.py                # Streamlit 入口
4
5  docs/                      # 存放设计文档的地方
6    example.txt
7
8  docforge_core/
9    config/
10     settings.py            # 环境变量、模型配置、路径配置
11
12   domain/
13     schemas.py             # Pydantic 数据结构
14     enums.py               # Literal / Enum
15     constants.py           # 默认 top_k、轮次、目录名
16
17   io/
18     run_paths.py           # run_id 目录管理
19     file_registry.py       # SourceItem 生成与文件落盘
20     state_store.py         # state.json 读写
21
22   parsers/
23     base.py
24     docx_parser.py
25     pdf_parser.py
26     markdown_parser.py
27     text_parser.py
28     image_parser.py        # MVP 可先 stub
29
30   llm/
31     base.py                 # LLMProvider 协议
32     qwen_provider.py
33     deepseek_provider.py
34
35   embedding/
36     base.py
37     jina_embedding_provider.py
```

```

38
39     evidence/
40         extractor.py                # Evidence Unit 生成
41         qdrant_store.py             # Qdrant 写入 / 检索 / filter
42
43     agents/
44         reference_style_agent.py
45         product_understanding_agent.py
46         software_diagnosis_agent.py
47         template_strategy_agent.py
48         outline_agent.py
49         writer_agent.py
50         screenshot_binding_agent.py
51         audit_agent.py
52
53     graph/
54         state.py                    # DocForgeState
55         nodes.py                    # LangGraph node function
56         router.py                   # 条件边
57         build_graph.py              # 编译工作流
58
59     gates/
60         plan_quality_gate.py
61         draft_quality_gate.py
62
63     exporters/
64         docx_exporter.py
65
66     prompts/
67         reference_style.md
68         product_understanding.md
69         writer.md
70         audit.md
71
72     data/
73         runs/
74             {run_id}/
75                 sources/
76                     reference/
77                     product/
78                     screenshots/
79                 parsed/
80                 evidence/
81                 drafts/
82                 audits/
83                 exports/
84                 state.json

```

```
85     qdrant/  
86  
87     tests/  
88         fixtures/  
89         unit/  
90         integration/  
91  
92     pyproject.toml  
93     README.md
```

技术栈

代码块

```
1  Python 3.11 / 3.12  
2  LangGraph  
3  Streamlit  
4  Qdrant Local Persistent Mode  
5  Jina-embeddings-v3  
6  QwenProvider / DeepSeekProvider  
7  python-docx  
8  pypdf / pymupdf  
9  Pydantic
```

重要的工程决策

决策项	结论	原因
数据模型	工程代码使用 Pydantic BaseModel，不只用 TypedDict	TypedDict 只做静态类型提示，运行时不校验；Pydantic 可以在运行时拦截错误字段和错误枚举，避免 source_type、file_type、corpus_type、allowed_usage 写错。
State 持久化	MVP 使用 state.json，不先上 SQLite	当前是本地单人 MVP，state.json 更直观、易调试、实现成本低；SQLite 会引入表结构、迁移和调试复杂度，收益不明显。
Qdrant 部署	MVP 使用 Qdrant Local Persistent Mode	不需要额外启动服务，不依赖 Docker，数据可持久化，足够支撑本地 MVP；等多人并发、服务化部署或证据量明显变大时，再切换 Docker Qdrant Server。
Agent 实现方式	MVP 阶段 LangGraph 节点先用普通 Python node function，复杂逻辑放 service class，不急抽象 BaseAgent 继承体系	当前最重要的是稳定 State、Evidence、FrozenDocPlan、AuditReport 等输入输出契约；过早设计 BaseAgent 继承体系容易造成过度抽象，后续改字段会牵一发动全身。
URL 能力	URL 字段预留，但 MVP 禁用	URL 读取会引入登录态、动态页面、反爬、网页截图、页面解析和证据可信度等复杂问题，会把 MVP 拖成网页采集系统；当前目标是先跑通软著文档生成闭环。

项目目录

代码块

```
1  docforge/
2    app/
3      main.py
4
5  docforge_core/
6    __init__.py
7
8  config/
9    __init__.py
10   settings.py
11
12  domain/
13    __init__.py
14    enums.py
15    schemas.py
16    constants.py
17
18  io/
19    __init__.py
20    run_paths.py
21    state_store.py
```

```
22     file_registry.py
23
24     parsers/
25         __init__.py
26         base.py
27         docx_parser.py
28         pdf_parser.py
29         markdown_parser.py
30         text_parser.py
31         image_parser.py
32
33     llm/
34         __init__.py
35         base.py
36         qwen_provider.py
37         deepseek_provider.py
38
39     embedding/
40         __init__.py
41         base.py
42         jina_embedding_provider.py
43
44     evidence/
45         __init__.py
46         extractor.py
47         qdrant_store.py
48
49     agents/
50         __init__.py
51         reference_style_agent.py
52         product_understanding_agent.py
53         software_diagnosis_agent.py
54         template_strategy_agent.py
55         outline_agent.py
56         writer_agent.py
57         screenshot_binding_agent.py
58         audit_agent.py
59
60     graph/
61         __init__.py
62         state.py
63         nodes.py
64         router.py
65         build_graph.py
66
67     gates/
68         __init__.py
```

```
69     plan_quality_gate.py
70     draft_quality_gate.py
71
72     exporters/
73         __init__.py
74         docx_exporter.py
75
76     prompts/
77         reference_style.md
78         product_understanding.md
79         writer.md
80         audit.md
81
82     data/
83         runs/
84         qdrant/
85
86     tests/
87         fixtures/
88         unit/
89         integration/
90
91     .env.example
92     .gitignore
93     pyproject.toml
94     README.md
```

工程依赖

代码块

```
1  [project]
2  name = "docforge"
3  version = "0.1.0"
4  description = "墨衡 DocForge - 软件著作权文档多智能体生成系统"
5  requires-python = ">=3.11,<3.13"
6  dependencies = [
7      "streamlit>=1.35.0",
8      "langgraph>=0.2.0",
9      "langchain-core>=0.2.0",
10     "pydantic>=2.7.0",
11     "pydantic-settings>=2.2.0",
12     "python-dotenv>=1.0.1",
13     "qdrant-client>=1.9.0",
14     "python-docx>=1.1.2",
```

```
15     "pypdf>=4.2.0",
16     "pymupdf>=1.24.0",
17     "markdown>=3.6",
18     "beautifulsoup4>=4.12.3",
19     "pillow>=10.3.0",
20     "httpx>=0.27.0",
21     "tenacity>=8.3.0",
22     "orjson>=3.10.0",
23 ]
24
25 [project.optional-dependencies]
26 dev = [
27     "pytest>=8.2.0",
28     "pytest-cov>=5.0.0",
29     "ruff>=0.4.0",
30     "mypy>=1.10.0",
31 ]
32
33 [tool.ruff]
34 line-length = 100
35 target-version = "py311"
36
37 [tool.pytest.ini_options]
38 testpaths = ["tests"]
39 pythonpath = ["."]
```

.env.example

代码块

```
1  DOCFORGE_ENV=local
2  DOCFORGE_DATA_DIR=data
3  DOCFORGE_QDRANT_PATH=data/qdrant
4
5  DEFAULT_LLM_PROVIDER=qwen
6  DEFAULT_EMBEDDING_PROVIDER=jina
7
8  QWEN_API_KEY=
9  QWEN_BASE_URL=
10 QWEN_MODEL=qwen-plus
11
12 DEEPSEEK_API_KEY=
13 DEEPSEEK_BASE_URL=
14 DEEPSEEK_MODEL=deepseek-chat
15
```

```
16 JINA_API_KEY=  
17 JINA_EMBEDDING_MODEL=jina-embeddings-v3  
18  
19 MAX_REVISION_ROUND=3  
20 DEFAULT_TOP_K=8
```

Sprint 拆分：

Sprint 0：工程骨架 + 基础配置

目标：

代码块

- 1 创建项目目录、pyproject.toml、环境变量配置、基础 README、data/runs 目录规则。

验收标准：

代码块

- 1 1. 项目可以本地启动；
- 2 2. pytest 可以运行；
- 3 3. Streamlit 空页面可以启动；
- 4 4. 所有路径通过 PathManager 生成，不手写散落路径；
- 5 5. .env.example 存在，但不提交真实 API Key。

Sprint 1：核心数据结构 + StateStore

目标：

代码块

- 1 用 Pydantic 定义 SourceItem、EvidenceItem、DocForgeState、FrozenDocPlan、AuditReport 等核心对象。

验收标准：

代码块

- 1 1. 可以创建一个 run_id；

2. 可以初始化 `state.json`;
3. 可以读取和更新 `state.json`;
4. `SourceItem` 区分 `source_type` 和 `file_type`;
5. URL 字段存在但 MVP 标记 `disabled`。

这个 Sprint 必须提前做。没有类型契约，后面 Cursor 写 Agent 会乱飞。

Sprint 2: Streamlit 上传界面 + 文件落盘

目标：

代码块

- 1 完成本地上传界面，支持参考资料、自有产品资料、截图分区上传。

验收标准：

代码块

1. 上传参考软著后落到 `sources/reference`;
2. 上传产品资料后落到 `sources/product`;
3. 上传截图后落到 `sources/screenshots`;
4. 每个文件生成 `SourceItem`;
5. `source_registry` 写入 `state.json`;
6. 不支持 URL 上传。

Sprint 3: 文档解析器

目标：

代码块

- 1 实现 `docx` / `pdf` / `md` / `txt` / `html` 基础解析，图片解析先 `stub`。

验收标准：

代码块

1. `docx` 可提取段落文本;
2. `pdf` 可提取页面文本;
3. `md` / `txt` / `html` 可提取文本;

- 4 4. 解析结果写入 `parsed/`;
- 5 5. `ParsedAsset` 写入 `state.json`;
- 6 6. 解析失败要记录 `parse_error`, 而不是直接崩。

Sprint 4: Provider Adapter + EmbeddingProvider

目标:

代码块

- 1 实现 `QwenProvider`、`DeepSeekProvider`、`JinaEmbeddingProvider` 的接口层。

验收标准:

代码块

- 1 1. `LLMProvider` 有统一 `generate_json` / `generate_text` 接口;
- 2 2. `EmbeddingProvider` 有 `embed_texts` 接口;
- 3 3. 所有 `Agent` 不直接调用厂商 SDK;
- 4 4. `MockProvider` 可用于单测;
- 5 5. `API Key` 只从环境变量读取。

这个 Sprint 必须在 `Qdrant Evidence` 写入前完成, 因为 `Evidence Unit` 生成和 `embedding` 都依赖 `Provider`。

Sprint 5: Evidence Unit 生成 + QdrantStore

目标:

代码块

- 1 把 `ParsedAsset` 转成 `EvidenceItem`, 并写入 `Qdrant`。

验收标准:

代码块

- 1 1. 每个 `EvidenceItem` 有 `corpus_type` / `allowed_usage`;
- 2 2. `reference` 资料只能生成 `reference_style` / `style_only`;
- 3 3. `product` 资料只能生成 `product_evidence` / `factual_evidence`;
- 4 4. `Qdrant collection` 使用 `docforge_{run_id}`;

- 5 5. Writer 查询必须带 product_evidence filter;
- 6 6. ReferenceStyle 查询必须带 reference_style filter;
- 7 7. top_k 默认 8。

Sprint 6：诊断、模板推荐、HumanConfirmGate、FrozenDocPlan

目标：

代码块

- 1 跑通从资料理解到模板确认，再到 FrozenDocPlan 冻结。

验收标准：

代码块

- 1 1. SoftwareDiagnosisAgent 输出 DiagnosisResult;
- 2 2. TemplateStrategyAgent 输出 TemplateStrategy;
- 3 3. Streamlit 展示推荐方案;
- 4 4. 用户可选择按 Agent 推荐或手动覆盖;
- 5 5. 用户确认后生成 FrozenDocPlan;
- 6 6. FrozenDocPlan.locked_status = locked;
- 7 7. 一级目录策略冻结。

Sprint 7：OutlineAgent + PlanQualityGate

目标：

代码块

- 1 基于 FrozenDocPlan 生成目录，并在写正文前做质量门禁。

验收标准：

代码块

- 1 1. OutlineAgent 不允许改一级目录;
- 2 2. PlanQualityGate 检查 10 项 checklist;
- 3 3. 不通过时最多生成 5~8 个问题;
- 4 4. 不能直接进入 WriterAgent;

5 5. 通过后 workflow_status = PLAN_GATE_PASSED。

Sprint 8: WriterAgent + ScreenshotBindingAgent

目标：

代码块

1 生成 v1 草稿结构化内容，并绑定截图。

验收标准：

代码块

1. WriterAgent 只输出结构化 JSON，不直接写 docx；
2. 每个章节带 evidence_ids；
3. 产品事实只引用 product_evidence；
4. 参考软著只影响写法，不进入事实引用；
5. ScreenshotBindingAgent 生成截图绑定关系；
6. draft_v1 写入 drafts/。

Sprint 9: AuditAgent + DraftQualityGate + v2/v3 循环

目标：

代码块

1 实现审计、扣分、修订循环、风险版判断。

验收标准：

代码块

1. AuditAgent 输出 AuditReport；
2. 支持 blocker / major / minor / suggestion；
3. $score = 100 - blocker_count * 30 - major_count * 8 - minor_count * 3$ ；
4. blocker 一票否决；
5. 最多 v1 / v2 / v3；
6. v3 不通过时选择最高质量草稿作为风险版来源；

7 7. audit_report 内部保存，不作为用户下载产物。

Sprint 10: DOCX ExportAgent + 端到端样例

目标：

代码块

1 将结构化草稿导出为《软件著作权文档》.docx 或风险版 DOCX。

验收标准：

代码块

1. 导出 A4；
2. 中文宋体，英文 Times New Roman；
3. 正文小四，1.5 倍行距；
4. 一级标题黑体三号；
5. 二级标题黑体四号；
6. 图片宽度不超过页面宽度；
7. 文件名区分最终版和风险版；
8. Streamlit 可下载最终 docx；
9. 完成一个端到端 demo。

构建项目（via 提示词，给IDE如Cursor）

1、创建项目骨架：

代码块

```
1 提示词 001：创建墨衡 DocForge 项目骨架
2
3 你现在是我的 Python 工程开发助手。请基于下面要求，为“墨衡 DocForge”创建项目骨架。
4
5 项目背景：
6 墨衡 DocForge 是一个面向软件著作权文档生成的多智能体文档生产系统。MVP 阶段使用 Python、
  LangGraph、Streamlit、Qdrant Local Persistent Mode、Pydantic、python-docx 实现。
7
8 当前只做 Sprint 0：项目骨架 + 环境配置。
9 不要实现 Agent 逻辑。
```

```
10 不要实现 Qdrant 写入。
11 不要实现文档解析。
12 不要实现 DOCX 导出。
13 不要实现 URL 读取。
14 不要写复杂业务逻辑。
15
16 请创建如下目录结构：
17
18 docforge/
19     app/
20         main.py
21
22     docforge_core/
23         __init__.py
24
25         config/
26             __init__.py
27             settings.py
28
29         domain/
30             __init__.py
31             enums.py
32             schemas.py
33             constants.py
34
35         io/
36             __init__.py
37             run_paths.py
38             state_store.py
39             file_registry.py
40
41     parsers/
42         __init__.py
43         base.py
44         docx_parser.py
45         pdf_parser.py
46         markdown_parser.py
47         text_parser.py
48         image_parser.py
49
50     llm/
51         __init__.py
52         base.py
53         qwen_provider.py
54         deepseek_provider.py
55
56     embedding/
```

```
57     __init__.py
58     base.py
59     jina_embedding_provider.py
60
61     evidence/
62         __init__.py
63         extractor.py
64         qdrant_store.py
65
66     agents/
67         __init__.py
68         reference_style_agent.py
69         product_understanding_agent.py
70         software_diagnosis_agent.py
71         template_strategy_agent.py
72         outline_agent.py
73         writer_agent.py
74         screenshot_binding_agent.py
75         audit_agent.py
76
77     graph/
78         __init__.py
79         state.py
80         nodes.py
81         router.py
82         build_graph.py
83
84     gates/
85         __init__.py
86         plan_quality_gate.py
87         draft_quality_gate.py
88
89     exporters/
90         __init__.py
91         docx_exporter.py
92
93     prompts/
94         reference_style.md
95         product_understanding.md
96         writer.md
97         audit.md
98
99     data/
100         runs/
101         qdrant/
102
103     tests/
```

```
104     fixtures/
105     unit/
106     integration/
107
108     .env.example
109     .gitignore
110     pyproject.toml
111     README.md
112
113 pyproject.toml 使用以下依赖：
114 - streamlit
115 - langgraph
116 - langchain-core
117 - pydantic
118 - pydantic-settings
119 - python-dotenv
120 - qdrant-client
121 - python-docx
122 - pypdf
123 - pymupdf
124 - markdown
125 - beautifulsoup4
126 - pillow
127 - httpx
128 - tenacity
129 - orjson
130
131 dev 依赖：
132 - pytest
133 - pytest-cov
134 - ruff
135 - mypy
136
137 .env.example 包含：
138 DOCFORGE_ENV=local
139 DOCFORGE_DATA_DIR=data
140 DOCFORGE_QDRANT_PATH=data/qdrant
141 DEFAULT_LLM_PROVIDER=qwen
142 DEFAULT_EMBEDDING_PROVIDER=jina
143 QWEN_API_KEY=
144 QWEN_BASE_URL=
145 QWEN_MODEL=qwen-plus
146 DEEPSEEK_API_KEY=
147 DEEPSEEK_BASE_URL=
148 DEEPSEEK_MODEL=deepseek-chat
149 JINA_API_KEY=
150 JINA_EMBEDDING_MODEL=jina-embeddings-v3
```



```
151 MAX_REVISION_ROUND=3
152 DEFAULT_TOP_K=8
153
154 app/main.py 只需要实现一个最小 Streamlit 页面:
155 标题: 墨衡 DocForge
156 说明: 软件著作权文档生成系统
157 显示当前为 Sprint 0 骨架版本
158 不要做文件上传
159
160 settings.py 使用 pydantic-settings 读取环境变量。
161 constants.py 放默认常量, 例如 DEFAULT_TOP_K = 8, MAX_REVISION_ROUND = 3。
162
163 README.md 简要说明:
164 1. 项目定位
165 2. MVP 技术栈
166 3. 当前 Sprint 只完成项目骨架
167 4. 如何启动 Streamlit
168
169 请确保:
170 1. 代码能通过基本 import;
171 2. pytest 可以运行;
172 3. 不要引入任何真实 API Key;
173 4. 不要写死绝对路径;
174 5. 所有路径使用 pathlib.Path;
175 6. 保持 Windows 11 + PowerShell 和 macOS 都能运行。
```

2、实现核心数据结构和 StateStore

代码块

```
1 提示词 002: 实现核心数据结构和 StateStore
2
3 继续开发“墨衡 DocForge”。
4
5 当前进入 Sprint 1: 核心数据结构 + StateStore。
6
7 目标:
8 1. 使用 Pydantic BaseModel 实现核心领域模型;
9 2. 实现 run_id 生成;
10 3. 实现 data/runs/{run_id}/ 目录创建;
11 4. 实现 state.json 初始化、读取、保存、更新;
12 5. 增加单元测试。
13
```

14 重要原则:

- 15 1. 设计文档里的 TypedDict 只是结构说明, 工程代码使用 Pydantic BaseModel。
- 16 2. SourceItem 必须区分 source_type 和 file_type。
- 17 3. source_type 表示业务来源类型, 例如 reference_soft_copyright_doc / prd / hld / screenshot。
- 18 4. file_type 表示文件格式, 例如 docx / pdf / md / txt / html / png / jpg。
- 19 5. corpus_type 和 allowed_usage 是证据隔离的生命线, 不能写错。
- 20 6. URL 相关字段只作为 Phase 2 预留, MVP 不启用。
- 21 7. Qdrant collection 名必须是 docforge_{run_id}。
- 22 8. state.json 是 MVP 阶段的业务状态持久化文件。
- 23 9. State 不保存完整原始文档、完整截图、完整 DOCX、大段原文。
- 24 10. 只保存路径、ID、结构化摘要和状态指针。

25

26 请在 docforge_core/domain/enums.py 中实现以下 Enum:

- 27 - WorkflowStatus
- 28 - NextAction
- 29 - SourceType
- 30 - FileType
- 31 - CorpusType
- 32 - AllowedUsage
- 33 - ParseStatus
- 34 - EvidenceType
- 35 - EvidenceStrength
- 36 - ConfirmationStatus
- 37 - ConfirmationType
- 38 - LockedStatus
- 39 - AuditSeverity
- 40 - AuditIssueType
- 41 - GateType
- 42 - ExportType

43

44 请在 docforge_core/domain/schemas.py 中实现以下 Pydantic 模型:

- 45 - SourceItem
- 46 - ParsedAsset
- 47 - ProductSourceProfile
- 48 - ReferenceStyleProfile
- 49 - ProductProfile
- 50 - ProductFact
- 51 - EvidenceItem
- 52 - DiagnosisResult
- 53 - TemplateStrategy
- 54 - HumanConfirmation
- 55 - FrozenDocPlan
- 56 - DocumentOutline
- 57 - SectionPlan
- 58 - ScreenshotBinding
- 59 - MissingInformationItem

```
60 - DraftVersion
61 - AuditIssue
62 - AuditReport
63 - QualityGateReport
64 - ExportResult
65 - StateTransitionLog
66 - DocForgeState
67
68 SourceItem 字段必须包含:
69 - source_id
70 - source_type
71 - file_type
72 - corpus_type
73 - allowed_usage
74 - file_name
75 - file_path
76 - url
77 - is_reference_source
78 - is_product_source
79 - uploaded_at
80 - parse_status
81 - parse_error
82 - notes
83 - metadata
84
85 SourceItem 校验规则:
86 1. 如果 corpus_type = reference_style, 则 allowed_usage 必须是 style_only;
87 2. 如果 corpus_type = product_evidence, 则 allowed_usage 必须是
    factual_evidence;
88 3. 如果 source_type = reference_soft_copyright_doc, 则 is_reference_source 必须
    为 true;
89 4. 如果 source_type = product_url, MVP 阶段允许模型存在, 但不得在默认流程中创建;
90 5. source_type 不允许填 docx、pdf、png、jpg;
91 6. docx、pdf、png、jpg 只能填到 file_type。
92
93 DocForgeState 默认值:
94 - workflow_status = CREATED
95 - next_action = ingest_materials
96 - max_revision_round = 3
97 - revision_round = 0
98 - plan_quality_gate_passed = false
99 - source_registry = []
100 - parsed_assets = []
101 - evidence_map = []
102 - draft_versions = []
103 - audit_reports = []
104 - errors = []
```

```
105 - warnings = []
106 - qdrant_collection = docforge_{run_id}
107
108 请在 docforge_core/io/run_paths.py 中实现:
109 - generate_run_id()
110 - get_run_dir(run_id)
111 - get_sources_dir(run_id)
112 - get_reference_dir(run_id)
113 - get_product_dir(run_id)
114 - get_screenshots_dir(run_id)
115 - get_parsed_dir(run_id)
116 - get_evidence_dir(run_id)
117 - get_drafts_dir(run_id)
118 - get_audits_dir(run_id)
119 - get_exports_dir(run_id)
120 - ensure_run_dirs(run_id)
121
122 路径规则:
123 data/runs/{run_id}/
124     sources/
125     reference/
126     product/
127     screenshots/
128     parsed/
129     evidence/
130     drafts/
131     audits/
132     exports/
133     state.json
134
135 请在 docforge_core/io/state_store.py 中实现 StateStore:
136 - create_initial_state(project_name: str | None = None) -> DocForgeState
137 - save_state(state: DocForgeState) -> Path
138 - load_state(run_id: str) -> DocForgeState
139 - update_state(run_id: str, **updates) -> DocForgeState
140
141 state.json 保存时:
142 1. 使用 UTF-8;
143 2. 保留中文;
144 3. JSON 缩进为 2;
145 4. 使用 Pydantic 的 model_dump 或 model_dump_json;
146 5. 读取时必须重新通过 Pydantic 校验。
147
148 请增加 tests/unit/test_state_store.py, 覆盖:
149 1. 可以创建初始 state;
150 2. 创建 state 时会生成 run 目录;
151 3. state.json 可以保存;
```

152 4. *state.json* 可以读取;
153 5. *qdrant_collection* = *docforge_{run_id}*;
154 6. *SourceItem* 中 *source_type* = *docx* 应该校验失败;
155 7. *SourceItem* 中 *file_type* = *docx* 应该校验通过;
156 8. *reference_style* 必须对应 *style_only*;
157 9. *product_evidence* 必须对应 *factual_evidence*;
158 10. URL 相关字段默认为空, 不参与 MVP 流程。
159
160 注意:
161 当前 Sprint 不要实现 Streamlit 上传。
162 当前 Sprint 不要实现 Agent。
163 当前 Sprint 不要实现 LangGraph 流程。
164 当前 Sprint 不要实现 Qdrant。
165 当前 Sprint 不要实现文档解析。
166 只实现数据契约和 StateStore。